

Guia de Estudo

Tecnologias Frontend

React, Vue e o ecossistema moderno

O que você vai encontrar

- Panorama de como o frontend moderno se organiza em camadas
- React e Vue em detalhe: conceitos, sintaxe, exemplos e boas práticas
- Comparação direta entre React e Vue
- Outros frameworks: Angular, Svelte, Solid e alternativas leves
- Meta-frameworks (Next.js, Nuxt, Astro...) e estratégias de renderização
- Ferramentas de apoio: TypeScript, Vite, estado, estilo e testes
- Como escolher a tecnologia certa para cada situação

Pré-requisitos

Este guia assume que você já tem base em HTML semântico, CSS (Flexbox/Grid) e JavaScript moderno (ES6+, módulos, async/await). Se algum desses pontos ainda estiver fraco, revise-o antes de avançar para os frameworks.

1. Panorama do frontend moderno

Hoje, construir uma interface web raramente significa escrever apenas HTML, CSS e JS à mão. O ecossistema se organiza em camadas, e cada tecnologia resolve um problema específico. Entender essas camadas evita a confusão comum de comparar coisas que não são do mesmo tipo (por exemplo, React com Next.js).

Camada	Para que serve	Exemplos
Base da web	Estrutura, aparência e comportamento	HTML, CSS, JavaScript
Linguagem tipada	Segurança e autocompletar em projetos grandes	TypeScript
Biblioteca / framework de UI	Componentes e atualização reativa da tela	React, Vue, Angular, Svelte, Solid
Meta-framework	Rotas, renderização no servidor, otimizações	Next.js, Nuxt, SvelteKit, Astro, Remix
Build e tooling	Empacotar, servir em dev, lint, formatação	Vite, esbuild, npm/pnpm, ESLint, Prettier
Estilo	Organizar CSS em escala	Tailwind, CSS Modules, Sass, styled-components
Estado e dados	Compartilhar estado e buscar dados da API	Redux Toolkit, Zustand, Pinia, TanStack Query
Testes	Garantir que a interface funciona	Vitest, Jest, Testing Library, Playwright

2. A ideia central: UI como função do estado

Todos os frameworks modernos compartilham a mesma ideia: em vez de manipular o DOM manualmente (`document.querySelector...`), você descreve **como a tela deve parecer para um determinado estado**, e o framework cuida de atualizar o DOM quando o estado muda. Isso é programação **declarativa**.

- **Componentes:** peças reutilizáveis de interface que juntam estrutura, estilo e lógica.
- **Props:** dados que o componente pai passa ao filho (fluxo unidirecional).
- **Estado:** dados internos que, ao mudar, provocam nova renderização.
- **Eventos:** o filho avisa o pai que algo aconteceu (cliques, envio de formulário).
- **Ciclo de vida:** montar, atualizar e desmontar um componente.

Guarde isto

Aprender React ou Vue é, em grande parte, aprender a pensar em componentes, estado e fluxo de dados. Esse raciocínio se transfere entre todos os frameworks.

3. React

Biblioteca criada pelo Facebook (Meta) e mantida por ela e pela comunidade. É a tecnologia de UI mais usada no mercado, com o maior ecossistema de bibliotecas e vagas. Documentação oficial: react.dev.

Conceitos-chave

- **JSX:** sintaxe que mistura HTML e JavaScript. O que você escreve como `<Botao />` vira uma chamada de função em JS.
- **Componentes de função:** funções que recebem props e retornam JSX. Componentes de classe ficaram para código legado.
- **Hooks:** funções que começam com `use` e dão estado e efeitos aos componentes de função.
- **Renderização:** quando o estado muda, o React executa o componente de novo e atualiza só o que mudou no DOM.
- **Imutabilidade:** nunca altere estado diretamente; crie um novo objeto ou array.

Principais hooks

Hook	Uso	Observação
<code>useState</code>	Guardar estado local	Atualização pode ser agrupada (batching)
<code>useEffect</code>	Sincronizar com algo externo (API, timers, eventos)	Nem todo cálculo precisa de effect; evite abusar
<code>useRef</code>	Guardar valor mutável ou acessar um elemento do DOM	Não dispara nova renderização
<code>useContext</code>	Ler valores compartilhados sem passar props	Útil para tema, usuário logado
<code>useMemo / useCallback</code>	Memorizar cálculos e funções	Use depois de medir; não é padrão
<code>useReducer</code>	Estado com muitas transições	Bom para lógica mais complexa

Exemplo: contador com efeito

```
import { useState, useEffect } from "react";

function Contador({ inicial = 0 }) {
  const [valor, setValor] = useState(inicial);

  useEffect(() => {
    document.title = `Cliques: ${valor}`;
  }, [valor]); // roda quando "valor" muda

  return (
    <button onClick={() => setValor(valor + 1)}>
      Cliques: {valor}
    </button>
  );
}
```

Exemplo: listas e renderização condicional

```
function ListaTarefas({ tarefas }) {
  if (tarefas.length === 0) return <p>Nada por aqui.</p>;

  return (
    <ul>
      {tarefas.map((t) => (
        <li key={t.id}>{t.titulo}</li>    // "key" é obrigatória
      ))}
    </ul>
  );
}
```

Erros comuns de iniciantes

- Esquecer a key em listas, ou usar o índice quando a lista muda de ordem.
- Mutar estado (`lista.push(x)`) em vez de criar uma nova cópia.
- Array de dependências errado no `useEffect`, causando loops ou dados velhos.
- Usar `useEffect` para tudo, quando bastaria calcular durante a renderização.
- Passar props por muitos níveis (prop drilling) em vez de usar contexto ou composição.

Novidades recentes

As versões atuais do React trazem Server Components, Actions para formulários e o React Compiler, que reduz a necessidade de `useMemo` e `useCallback` manuais. Confira o estado atual de cada recurso em react.dev, pois eles evoluem rápido.

4. Vue

Framework progressivo criado por Evan You, mantido por uma comunidade independente. Conhecido pela curva de aprendizado suave e pela documentação excelente (vuejs.org). Você pode adotar aos poucos numa página existente ou construir uma aplicação completa.

Conceitos-chave

- **Single-File Components (.vue):** um arquivo com três blocos: `<template>`, `<script>` e `<style>`.
- **Reatividade automática:** o Vue rastreia quais dados cada parte da tela usa e atualiza apenas o necessário, sem você declarar dependências.
- **Composition API:** forma moderna de organizar lógica com `ref`, `computed` e `watch`. A Options API (`data`, `methods`, `computed`) continua válida.
- **Diretivas:** atributos especiais no template que ligam HTML à lógica.
- **Estilo com escopo:** `<style scoped>` isola o CSS do componente.

Principais diretivas

Diretiva	Função	Exemplo
v-bind (:)	Ligar atributo a um valor	:disabled="carregando"
v-on (@)	Escutar eventos	@click="salvar"
v-model	Ligação bidirecional em inputs	v-model="nome"
v-if / v-else	Renderização condicional	v-if="logado"
v-show	Alterna visibilidade via CSS	v-show="aberto"
v-for	Repetir elementos de uma lista	v-for="t in tarefas" :key="t.id"

Exemplo: contador com Composition API

```
<script setup>
import { ref, computed } from "vue";

const valor = ref(0); // estado reativo
const dobro = computed(() => valor.value * 2);
</script>

<template>
  <button @click="valor++">
    Cliques: {{ valor }} (dobro: {{ dobro }})
  </button>
</template>

<style scoped>
button { padding: 8px 16px; }
</style>
```

Ecossistema oficial

- **Vue Router:** navegação entre páginas.
- **Pinia:** gerenciamento de estado global (sucessor do Vuex).
- **Vite:** ferramenta de build criada pelo mesmo autor do Vue e hoje usada em todo o mercado.
- **Nuxt:** meta-framework para SSR, rotas por arquivos e mais.
- **Vue DevTools:** extensão para inspecionar componentes e estado.

Ponto de atenção

Em JavaScript, `ref` exige `.value` no script, mas não no template. É o tropeço mais comum de quem está começando.

5. React x Vue lado a lado

Aspecto	React	Vue
Natureza	Biblioteca focada em UI; o restante você escolhe	Framework progressivo com peças oficiais (router, Pinia)
Sintaxe	JSX: HTML dentro do JavaScript	Templates HTML com diretivas em arquivos .vue
Reatividade	Re-executa o componente; você gerencia dependências de hooks	Rastreamento automático de dependências
Estado local	useState / useReducer	ref / reactive
Estado global	Redux Toolkit, Zustand, Context	Pinia
Curva de aprendizado	Média: JSX e hooks exigem prática	Mais suave para quem vem de HTML/CSS
Mercado (vagas)	Maior volume de vagas e bibliotecas	Menor, mas sólido; forte em Europa e Ásia
Meta-framework	Next.js, React Router (Remix)	Nuxt
Mobile	React Native	Ionic / Capacitor (híbrido)
Governança	Meta e comunidade	Comunidade e sponsors independentes

Mesmo problema, duas formas

Campo de texto ligado a uma variável, mostrando o valor digitado:

```
// React
const [nome, setNome] = useState("");
<input value={nome} onChange={(e) => setNome(e.target.value)} />
<p>Olá, {nome}</p>

<!-- Vue -->
const nome = ref("");
<input v-model="nome" />
<p>Olá, {{ nome }}</p>
```

Conclusão prática

Nenhum dos dois é “melhor” em termos absolutos. React dá mais liberdade (e mais decisões a tomar); Vue dá mais convenções prontas. Os conceitos que você aprende em um valem quase integralmente no outro.

6. Outros frameworks e alternativas

Tecnologia	Ideia principal	Quando considerar
Angular	Framework completo da Google, com TypeScript, injeção de dependência, roteador e formulários embutidos; usa signals para reatividade	Grandes empresas e equipes que valorizam estrutura rígida e padrões uniformes
Svelte	Compilador: transforma componentes em JS enxuto no build, sem runtime pesado; reatividade por runes nas versões recentes	Projetos que priorizam desempenho, bundle pequeno e sintaxe concisa
Solid	Reatividade fina com signals e JSX, sem Virtual DOM	Quando quer sintaxe parecida com React e desempenho máximo
Preact	Alternativa de ~3 KB compatível com a API do React	Widgets e páginas em que o tamanho do bundle é crítico
Alpine.js	Interatividade leve direto no HTML com atributos	Sites tradicionais que precisam de pequenos toques dinâmicos
htmx	Troca de trechos de HTML via atributos, com o servidor renderizando	Apps orientadas a servidor que querem evitar SPA
Web Components	Padrão nativo do navegador para elementos customizados	Bibliotecas de componentes usadas em vários frameworks

Virtual DOM x compilação x signals

- **Virtual DOM (React, Vue):** mantém uma árvore em memória, compara com a anterior e aplica só as diferenças no DOM real.
- **Compilação (Svelte):** o framework analisa seu código no build e gera atualizações diretas no DOM.
- **Signals (Solid, Angular, Vue por dentro):** valores reativos que notificam exatamente quem depende deles, sem re-executar componentes inteiros.

7. Meta-frameworks e estratégias de renderização

Um framework de UI sozinho cuida apenas dos componentes. Para produção, normalmente você precisa de roteamento, carregamento de dados, SEO e desempenho. É aí que entram os meta-frameworks.

Meta-framework	Base	Destaques
Next.js	React	App Router, Server Components, SSR/SSG/ISR, grande ecossistema
React Router (Remix)	React	Foco em padrões web, loaders e actions para dados e formulários
Nuxt	Vue	Rotas por arquivos, auto-imports, módulos, SSR e SSG
SvelteKit	Svelte	Rotas, SSR, adaptadores para várias plataformas de deploy
Astro	Várias	Sites de conteúdo; envia pouco ou nenhum JS ao navegador

Formas de renderizar

Sigla	Nome	Como funciona	Bom para
CSR	Client-Side Rendering	O navegador baixa JS e monta a página	Painéis, apps atrás de login
SSR	Server-Side Rendering	O servidor gera o HTML a cada requisição	Conteúdo dinâmico com SEO
SSG	Static Site Generation	HTML gerado no build e servido pronto	Blogs, documentação, marketing
ISR	Incremental Static Regeneration	Páginas estáticas regeneradas sob demanda	Catálogos e conteúdos que mudam pouco
Hidratação	Hydration	JS “ativa” o HTML já entregue pelo servidor	Complemento de SSR e SSG

8. Ferramentas de apoio

TypeScript

Superset do JavaScript com tipos estáticos. Pega erros antes de rodar, melhora o autocompletar e documenta o código. Hoje é praticamente padrão em projetos profissionais de React, Vue e Angular.

```
type Tarefa = { id: number; titulo: string; feita: boolean };

function concluir(t: Tarefa): Tarefa {
  return { ...t, feita: true };
}
```

Build, gerenciadores de pacote e qualidade

- **Vite:** servidor de desenvolvimento muito rápido e build otimizado; padrão para React, Vue e Svelte.
- **npm, pnpm, yarn:** instalam e versionam dependências (package.json e lockfile).
- **ESLint e Prettier:** encontram problemas e padronizam o formato do código.
- **Git hooks (Husky, lint-staged):** rodam verificações antes de cada commit.

Estilo

Abordagem	Ideia	Ponto de atenção
Tailwind CSS	Classes utilitárias direto no HTML/JSEX	Markup mais carregado; exige convenções de equipe
CSS Modules	CSS local a cada componente, sem novas sintaxes	Menos ferramentas de tema prontas
Sass/SCSS	CSS com variáveis, aninhamento e mixins	Hoje o CSS nativo já cobre parte disso
CSS-in-JS	Estilos escritos em JavaScript	Custo em runtime; perdeu espaço em SSR
Bibliotecas de UI	Componentes prontos (shadcn/ui, MUI, Vuetify, PrimeVue)	Personalização e peso do bundle

Estado e dados

- **Estado local primeiro:** comece com o estado do próprio componente e só “suba” o estado quando necessário.
- **Estado global:** Redux Toolkit ou Zustand (React); Pinia (Vue).
- **Dados do servidor:** TanStack Query (React, Vue, Svelte e outros) cuida de cache, recarregamento e estados de carregamento e erro.
- **Formulários:** React Hook Form, VeeValidate, com Zod ou Valibot para validação.

Testes

Tipo	Ferramentas	O que verifica
Unitários	Vitest, Jest	Funções e lógica isolada
Componentes	Testing Library (React, Vue)	Comportamento visto pelo usuário
End-to-end	Playwright, Cypress	Fluxos completos no navegador
Acessibilidade	axe, Lighthouse, testes manuais	Uso com teclado e leitor de tela

9. Como escolher a tecnologia

Não existe resposta universal. Use estas perguntas para decidir:

Situação	Caminho comum
Quer o maior mercado de trabalho e ecossistema	React + TypeScript + Next.js
Equipe vindo de HTML/CSS, quer produtividade rápida	Vue + Vite (ou Nuxt)
Empresa grande, muitos times, padrões rígidos	Angular
Site de conteúdo (blog, docs, marketing)	Astro, ou Next.js/Nuxt com SSG
Foco em desempenho e bundle pequeno	Svelte/SvelteKit ou Solid
App interno atrás de login, sem SEO	SPA com Vite (React ou Vue)
Site tradicional com pouca interatividade	HTML + Alpine.js ou htmx

Critérios além da tecnologia

- **Conhecimento da equipe:** um framework que todos dominam vale mais do que o “melhor” que ninguém conhece.
- **Ecossistema:** existem bibliotecas, tutoriais e pessoas disponíveis para contratar?
- **Manutenção:** o projeto tem histórico estável e atualizações frequentes?
- **Requisitos do produto:** SEO, desempenho em celulares fracos, acessibilidade, offline.
- **Prazo:** o custo de aprender algo novo entra na conta.

10. Glossário rápido

Termo	Significado
SPA	Single Page Application: uma página que troca o conteúdo sem recarregar o navegador
Virtual DOM	Representação em memória da interface, usada para calcular o mínimo de mudanças no DOM
Hidratação	Processo de tornar interativo um HTML que veio pronto do servidor
Bundle	Arquivo JS/CSS final gerado no build e enviado ao navegador
Tree shaking	Remoção de código não utilizado no build
HMR	Hot Module Replacement: atualização instantânea no navegador durante o desenvolvimento
Prop drilling	Passar props por muitos níveis só para chegar a um componente distante
Reatividade	Capacidade de a interface reagir automaticamente a mudanças nos dados
Composable / Hook	Função reutilizável que encapsula lógica com estado (Vue / React)
Meta-framework	Framework construído sobre outro para adicionar rotas, SSR e otimizações
Signals	Valores reativos que notificam apenas os pontos que dependem deles
Monorepo	Um único repositório com vários projetos ou pacotes

Referências para continuar

Recurso	Endereço	Conteúdo
MDN Web Docs	developer.mozilla.org	Base de HTML, CSS e JavaScript
React	react.dev	Tutorial e referência oficial
Vue	vuejs.org	Guia, tutorial interativo e API
Angular	angular.dev	Documentação e tutoriais oficiais
Svelte	svelte.dev	Tutorial interativo
TypeScript	typescriptlang.org	Handbook e playground
Vite	vite.dev	Guia de build e configuração
web.dev	web.dev	Desempenho, acessibilidade e boas práticas

Nota: versões e recursos de frameworks mudam com frequência. Ao iniciar um projeto real, confirme o estado atual na documentação oficial de cada tecnologia.